

Stoquify Module System: Current State and Enterprise Control-Plane Proposal

Date: 2026-07-14

Repository: `E:\ohada_saaS\Focused_projects\stoquify`

Assessment mode: Read-only architecture, security, product, data, UI/UX, and release-governance review

Runtime changes made: None

Executive Verdict

Stoquify does not currently have a complete commercial module-management system. It has a credible module-governance foundation:

- a typed 20-entry module catalog;
- route, permission, owner, risk, status, and dependency metadata;
- a tenant-level entitlement evaluator;
- an administrator-facing Module Control Center;
- shared page, action, and API guard seams;
- audit logging for would-block decisions;
- a surface inventory with API coverage and baseline ratchet support;
- focused tests protecting important module-control rules.

Those foundations are real and useful. They are not yet a durable package, subscription, entitlement, billing, provisioning, deactivation, or cross-surface enforcement control plane.

The most important current finding is a control-state inconsistency. `MODULE_CONTROL_MODE` is declared as `observe`, and the Module Control Center says hard enforcement is off, but selected production pages and actions explicitly request `mode: "enforce"`. The shared API guard also requests enforce mode. Static verification found 55 explicit enforce call sites across production actions, 10 page-level enforce call sites, and four API routes using the enforcing module-access helper. These are concentrated in payroll, public receipt-token administration, and selected tenant APIs. Other surfaces explicitly remain in observe mode. Therefore, the current system is neither globally observe-only nor coherently pilot-enforced. It is mixed-mode, while the UI and several July 12 documents describe it as observe-only.

That inconsistency should be corrected before new package or billing behavior is connected. The first enterprise step is not broader enforcement. It is one authoritative enforcement policy that can truthfully report which modules, surfaces, tenants, and cohorts are observing, enforcing, or rolled back.

The recommended destination is a Module Control Plane with six clear ownership boundaries:

1. A versioned canonical module and surface registry.
2. Durable package, subscription, entitlement, dependency, and lifecycle records.
3. A provider-independent billing and provisioning boundary.
4. One server-side module access decision service used by every protected surface.

5. A service-owned Module Workbench read model and role-aware dashboard experience.
6. Append-only lifecycle evidence, observability, release gates, and tested rollback.

Stoquify can reach that destination incrementally. A safe sequence is: reconcile the current enforcement policy, ratchet the surface registry, add additive schema, migrate in observe mode, implement package and provisioning services, add lifecycle-safe UI states, then promote one bounded module surface at a time.

Assessment Method

The review was divided into architecture, security/RBAC, database and billing lifecycle, SaaS product packaging, enterprise dashboard UX, and release-assurance lanes. The final conclusions were synthesized against live repository evidence.

Evidence precedence was:

1. Current working-tree code and `prisma/schema.prisma`.
2. A fresh module surface inventory generated on 2026-07-14.
3. Current tests and package scripts.
4. Module proposals and architecture decisions dated 2026-07-12.
5. Graph reports only as secondary context.

The graph artifacts were not treated as current architectural truth. `graphify-out/` is dated 2026-07-02 and its manifest points to `E:\ohada saas\newStockFlow\aqstoqflow`, not this `Focused projects\stoquify` checkout. It can suggest broad historical patterns, but it cannot prove current file relationships in this repository.

Evidence Baseline

Fresh inventory

The live scanner was run in report mode and in warn/ratchet mode with output isolated from the tracked project reports.

Measure	2026-07-14 result
Catalog modules	20
Inventoried surfaces	318
Actions	112
Pages	109
Navigation entries	72
Layouts	10
API routes	10
API evidence records	2
Module services	3
Mapped records	271

Measure	2026-07-14 result
Unmapped records	34
Missing-permission records	16
Enforcement candidates	305
Baseline active gaps	55
Current active gaps	55
New gaps against baseline	0
Resolved gaps against baseline	0
Warn-mode ratchet	Passed

The 34 unmapped records are 33 actions and one module-service record. The 16 missing-permission findings are 15 actions and one page. Some findings are genuine ownership gaps; others are parser artifacts or shared helpers that need explicit classification. The scanner itself demonstrates this limitation by extracting malformed permission text from some error-handling branches. Counts are governance signals, not a substitute for code review.

The inventory also defaults discovered surfaces to `report-only`; it does not currently reconcile that label with explicit runtime `mode: "enforce"` declarations. Its coverage count and its effective-enforcement count are therefore different questions.

Focused verification

Verification	Result
<code>services/modules/__tests__/module-entitlement.service.test.ts</code>	Passed, 6 tests
<code>actions/modules/__tests__/module-control.actions.test.ts</code>	Passed, 3 tests
<code>scripts/__tests__/module-surface-inventory.test.js</code>	Passed, 10 tests
Isolated report-mode inventory	Passed, 318 records
Isolated warn-mode baseline ratchet	Passed, zero new gaps

Typecheck, the full policy gate, the full test suite, and application build were not run because this execution created documentation only and the worktree already contains extensive unrelated changes.

Confirmed Current System

Capability classification

Capability	Current classification	Evidence and honest interpretation
Canonical module slug list	Implemented	<code>services/modules/module-control-contracts.ts</code> defines 20 slugs.
Catalog metadata	Implemented	<code>module-catalog.service.ts</code> owns display data, routes, permissions, risk, core flag, aliases, and limited dependencies.
Tenant entitlement evaluation	Implemented but legacy-derived	<code>module-entitlement.service.ts</code> evaluates status and dependencies, but live data comes from <code>Organization.requestedModules</code> or legacy full-suite defaults.
Explicit entitlement input	Implemented as an in-memory evaluator option	No durable entitlement table feeds it.
Observe decisions	Implemented	Default evaluator mode is observe and would-block decisions can be logged.
Hard enforcement	Partially implemented and inconsistent	Payroll, receipt-token administration, and API helper paths explicitly enforce; other areas observe. The global UI reports hard enforcement off.
Module Control Center	Implemented diagnostic UI	<code>/dashboard/settings/modules</code> is permission-protected and shows catalog, state, source, risk, reason, and dependency gaps. It has no package or lifecycle commands.
Sidebar module metadata	Implemented metadata only	Sidebar entries carry <code>moduleSlug</code> , but visibility is filtered by RBAC permissions, not tenant entitlement.
Durable module entitlement source	Missing	<code>Organization.requestedModules String[]</code> is onboarding intent, not a commercial lifecycle record.
Package catalog	Proposal only	Ten package candidates are documented, but there are no package tables or services.
Subscription truth	Missing	No <code>TenantSubscription</code> model or service exists.
Billing-provider boundary	Missing for modules	No module billing adapter, event inbox, reconciliation, dunning, or provisioning service exists.
Activation/deactivation lifecycle	Proposal only	Status vocabulary exists, but no durable commands or state machine owns it.

Capability	Current classification	Evidence and honest interpretation
Entitlement lifecycle evidence	Partial	Would-block decisions write best-effort <code>AuditLog</code> rows; grants, suspensions, trials, overrides, billing changes, and rollback are not first-class immutable events.
Surface inventory	Implemented and improving	Scanner supports report, warn, and fail modes plus API evidence. Current package and CI wiring remains report-oriented.
Cross-surface leakage control	Partial and mostly proposed	Reports, exports, jobs, webhooks, proof views, BI cards, and schedulers are not first-class module registry records.
Release ratchet	Implemented but not promoted	Baseline comparison and tests exist; the module ratchet is not wired into <code>policy:gates</code> or CI.
Session module claims	Implemented but non-authoritative	<code>modulesEnabled</code> is hard-coded in <code>lib/security/auth-session.ts</code> .

What Is Strong Today

1. Vocabulary and ownership have a real home

The catalog is more than a sidebar list. It contains enough product and technical metadata to seed a proper registry. Alias normalization also gives migration logic a controlled path from registration labels such as `POS`, `OHADA accounting`, or `payment recon` to canonical slugs.

2. RBAC and tenant entitlement are conceptually separate

The evaluator records whether wildcard RBAC is present but does not allow wildcard permission to erase a module decision. This is the correct enterprise principle: the tenant must own the module, and the user must separately have permission to perform the operation.

3. The evaluator already understands useful lifecycle states

The contracts include active, trial, read-only, suspended, expired, unavailable, legacy-default, and system-default states. Read-only behavior distinguishes read surfaces from write, export, and job intent. This vocabulary should be preserved and made durable.

4. Shared server-side seams exist

`protect({ module })`, `requireApiModuleAccess()`, and direct page evaluation prove that server-side enforcement does not need to be invented from nothing. The next step is to centralize their policy and response contract.

5. The control-center UI is an honest starting point

The administrator page clearly labels registration intent, unknown requested modules, would-block state, risk, owner, source, reason, and dependency gaps. It is useful for diagnosis and can evolve into a Workbench without changing the dashboard's visual language.

6. The repository has a credible release-gate culture

Stoquify already uses policy and evidence gates across accounting, payment, inventory, purchasing, payroll, identity, migration safety, and release secrets. A module ratchet can fit that culture once its baseline, coverage, and pilot policy are ready.

Critical Current Gaps

1. The reported mode is not the effective mode

`MODULE_CONTROL_MODE` is `observe`, but callers can pass `mode: "enforce"`. The shared action wrapper defaults a provided module gate to enforce when the caller omits mode, and the API helper explicitly enforces. Payroll pages and actions also explicitly enforce. Meanwhile, `hardEnforcementEnabled` is always returned as `false`, and the Module Control Center says nothing is hard-blocked.

This is a governance and support risk. Operators cannot reliably answer whether a tenant is being observed or denied. It also makes rollback and release evidence ambiguous.

2. Registration intent is still runtime entitlement input

`Organization.requestedModules` cannot represent package version, price, subscription, trial dates, grace period, manual override, suspension, read-only retention, source precedence, or historical changes. Tenants with no requested modules receive legacy full-suite access. This preserves compatibility but is not commercial truth.

There are additional evaluator edge cases that matter before persistence: an empty explicit-entitlement array falls back to legacy derivation because only a non-empty array is accepted as explicit truth; a missing organization also falls into the empty-list legacy full-suite path; `startsAt` and `endsAt` are carried in the type but not evaluated; duplicate grants resolve by input order; required dependencies are checked for presence rather than effective active status; and read-only action/API reads are not treated like page/report reads. These are safe to fix in shadow mode but unsafe to leave ambiguous before broad enforcement.

3. Package and billing architecture is documentation-only

There is no `services/billing/` boundary and no package, subscription, entitlement, provisioning, dunning, or reconciliation service. Existing payment-provider models are payment-domain evidence and should not be reused as subscription truth.

4. Dependency declarations are too weak and contain wiring defects

Only required and recommended dependency types exist in code. Important technical, commercial, evidence, reporting, write-target, and retention relationships are absent. `cash_drawer` currently calls `dependenciesFor("pos")`, which gives it POS's dependency on sales rather than a direct cash-drawer dependency declaration. `compliance` calls `dependenciesFor("accounting")`, which currently returns no dependency for compliance.

Registration defaults also select POS and payment reconciliation without selecting all of their required Sales and Finance dependencies. The registration picker includes `RBAC roles`, which has no canonical module alias and becomes unknown module intent. Payroll's catalog dependency on accounting is only recommended, while payroll readiness code can treat missing or unready accounting as a blocker. Product, catalog, onboarding, and domain readiness therefore do not yet share one dependency truth.

5. Audit evidence is incomplete

Only would-block decisions are written, failures to write are logged and swallowed, and the event action remains `MODULE_ENTITLEMENT_OBSERVED` even when the decision mode is enforce. `AuditLog` is useful compatibility evidence, but it is not an append-only entitlement lifecycle ledger or tamper-evident evidence chain.

The current relation can also be mutated by ordinary application code and is configured to cascade when an organization is deleted. Commercial subscription and entitlement evidence should survive tenant archival under a legal retention policy, with database-level mutation protection for append-only records.

6. The surface registry is still mostly source-text inference

The scanner now covers routes, actions, navigation, layouts, module services, and APIs. It does not yet own reports, exports, jobs, webhooks, proofs, BI cards, public token outputs, scheduled processors, or cross-module source dependencies as explicit contracts. A passing no-new-gap ratchet means the baseline did not worsen; it does not mean the baseline is clean.

7. UI truth is permission-led

The sidebar filters by RBAC. The Module Control Center renders one card per module and has no package source, allowed intents, audit timeline, activation workflow, downgrade impact, reconciliation state, or detail drawer. Normal-user unavailable/read-only states are not uniformly service-owned across direct URLs.

8. Session claims can become stale or misleading

The hard-coded `modulesEnabled` list does not match the 20-entry catalog and is not derived from entitlement truth. It must not be used for authorization.

Adjacent Security Blockers

These findings are not all defects in the module evaluator itself, but they must be resolved before a privileged Module Workbench, tenant override workflow, or broad entitlement enforcement can be considered enterprise-ready.

P0. Wildcard roles currently enable cross-tenant POS terminal access

`RbacContext.isSuperUser` is derived from the wildcard `*` permission. New organization administrator roles receive `*`. POS terminal management accepts a caller-supplied organization identifier and permits a different tenant whenever `ctx.isSuperUser` is true; its focused test explicitly approves that path. This turns an ordinary tenant administrator role into a cross-tenant operator for that surface.

Remove tenant switching from ordinary wildcard roles. Tenant scope should come from the authenticated session or a separately authenticated platform-operator context. Any support access across tenants should use an explicit, short-lived, reason-bound, audited break-glass workflow with approval and revocation, not `isSuperUser`.

P0. Invitation role assignment lacks a grant ceiling

`sendInvite` requires fresh auth and `users.invite`, then accepts any role identifier belonging to the same organization. The workflow verifies tenant ownership of the role but does not require a separate role-assignment permission, compare the target role's authority with the actor's grant ceiling, or require maker-checker approval for wildcard/privileged roles. An inviter may therefore be able to create a more privileged user than intended.

Add an explicit `users.roles.assign` control, prohibit assignment above the actor's grant ceiling, require step-up authentication for privileged roles, and use maker-checker approval for wildcard or platform-sensitive roles. Test horizontal, vertical, stale-role, and concurrent-change cases.

P0. Fresh-auth evidence is session age, not verified step-up age

`requireFreshAuth` computes freshness from session creation time. When the session creation timestamp is absent, the claim defaults to the current time. This is not reliable proof that a password, passkey, or MFA challenge was completed recently. It is inadequate for package changes, entitlement overrides, tenant suspension, break-glass, or privileged role assignment.

Persist a server-verifiable step-up event timestamp and authentication method, bind it to the session and intended risk class, expire it quickly, and fail closed when evidence is absent. Sensitive services must consume this evidence rather than caller-supplied timestamps.

P1. Private-upload delivery needs a deployment-level access test

The upload action writes files beneath `public/uploads/<organizationId>` and returns `/uploads/<organizationId>/<file>`. A protected API route also serves files with a public cache directive. The source layout strongly suggests that static file delivery may bypass the protected API in common deployments, but this must be verified against the actual production build and proxy/CDN configuration before being stated as a confirmed exploit.

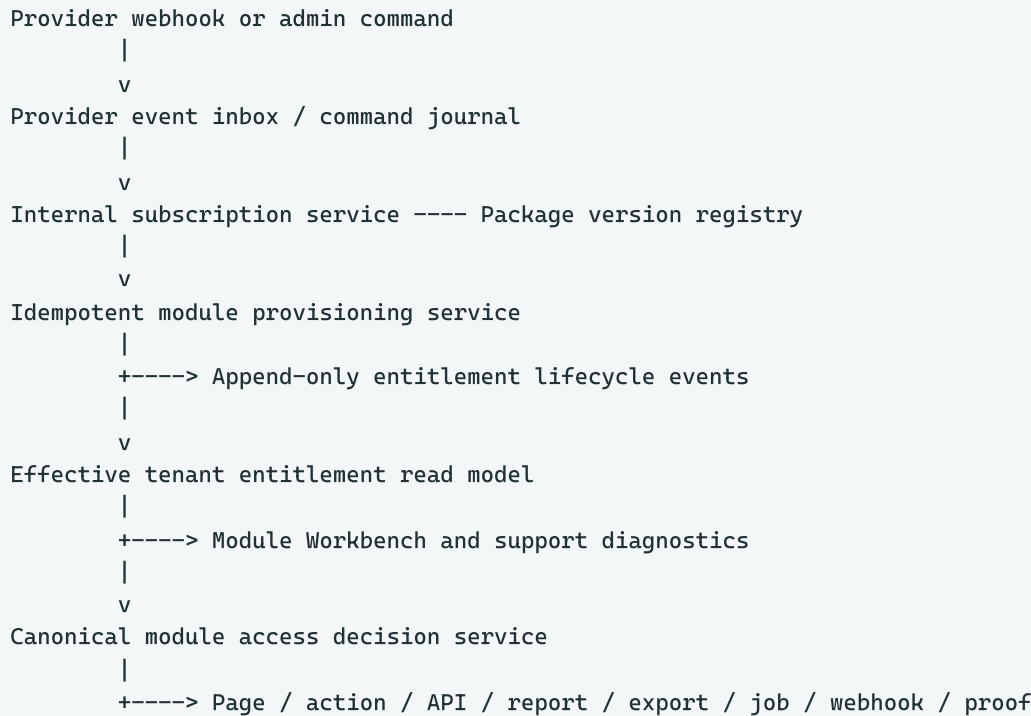
Move sensitive tenant files outside the public tree, serve them only through an authenticated tenant-scoped handler or short-lived signed URL, use private cache directives, and add direct static-path, cross-tenant, CDN-cache, and revoked-session tests.

P1. Tenant isolation remains application-convention dependent

The repository uses a global Prisma client and application-layer scoping. The reviewed terminal path demonstrates how one permissive exception can cross a tenant boundary. Before privileged module administration is introduced, high-risk services should use centrally scoped repositories or database-enforced tenant policies where practical, with negative cross-tenant tests as release gates.

Target Architecture

The target should have one-way ownership:



RBAC, tenant scope, fresh auth, maker-checker, and redaction remain separate inputs to the access decision. The sidebar is only a presentation consumer.

Source-of-truth rules

- Code-owned manifest: canonical slugs, stable module identity, technical owner, default risk, and static route hints.
- Database-owned commercial state: package versions, subscription assignments, grants, constraints, trials, overrides, retention, and lifecycle events.
- Service-owned effective truth: one computed entitlement decision per tenant, module, surface, and intent.
- Registry-owned surface truth: owner module, source modules, permission, risk, intent, guard, unavailable behavior, and rollback policy.
- UI-owned presentation only: filters, badges, drawers, and next-action affordances.
- Provider-owned evidence only: external payment/subscription event facts, never direct authorization.

Proposal 1: Reconcile Enforcement Policy Before Expansion

Business use case. Give engineering, support, security, and customers one truthful answer about whether a surface observes, enforces, or is in rollback.

Benefiting roles. Platform owner, security lead, support operator, release manager, module owner, and tenant administrator.

Confirmed current behavior. Global mode and UI say observe/hard-off, while selected payroll, receipt-token, and API paths explicitly enforce.

Truth owner. A new `module-enforcement-policy.service.ts`, backed by a versioned policy registry and release configuration.

Boundary. Replace caller-selected free-form mode with `resolveModuleEnforcementPolicy({ moduleSlug, surfaceKey, organizationId, intent })`. Callers may name a registered pilot, but they should not decide policy themselves. Return `observe`, `enforce`, or `rollback_observe` plus policy version, cohort, and reason.

UI pattern. A command brief showing effective mode, policy version, active pilots, rollback owner, and last change. Use an action queue for unregistered enforce call sites.

Value. Removes misleading control state, enables safe rollout, and gives support actionable diagnostics.

Risks and tradeoffs. Centralization adds a critical dependency. Cache and policy failures must fail according to a documented rule. A default-deny policy is inappropriate until durable entitlements and complete surface coverage exist; default observe is safer during migration.

Implementation path. Inventory every explicit enforce/observe call; freeze new direct mode declarations; add a policy resolver; make `hardEnforcementEnabled` computed rather than hard-coded; register existing enforced payroll/POS/API surfaces as explicit reviewed pilots or return them to observe; add a kill switch per pilot.

Verification. Contract tests for every mode, cohort, and rollback path; static gate rejecting new direct mode strings; support-view test; deny and rollback smoke tests; proof that non-pilot tenants are unaffected.

Proposal 2: Durable Entitlement and Package Schema

Business use case. Represent what a tenant purchased, why access exists, when it changes, and what remains after downgrade.

Benefiting roles. Tenant owner, finance, sales, support, security, auditors, product management, and engineering.

Confirmed current behavior. Only `requestedModules` is durable. Explicit entitlements exist only as evaluator input types.

Truth owner. `tenant-entitlement.service.ts` for effective access; `tenant-subscription.service.ts` for commercial lifecycle; a versioned module/package registry for definitions.

Boundary. Additive models should include `CommercialModule`, `CommercialModuleDependency`, `ModulePackage`, `ModulePackageVersion`, `ModulePackageVersionModule`, `TenantSubscription`, `TenantModuleGrant`, `TenantModuleConstraint`, `ModuleEntitlementEvent`, and migration-run evidence.

Use foreign keys to stable module IDs instead of relying only on free-form slug strings. Keep a non-null deterministic `sourceKey` or idempotency key for every grant; a nullable `sourceRef` in a composite unique key can still permit duplicate null-source rows in PostgreSQL. Package composition and price must be versioned so historical subscriptions do not change when a package is edited. Tenant-owned cross-table references should use composite tenant foreign keys, such as subscription plus organization, so an entitlement for one tenant cannot reference another tenant's subscription even if a service check is missed.

Represent commercial status, dunning stage, reconciliation exception, and access effect as separate concepts. Avoid contradictory combinations such as status `active` plus `readOnly = true` plus `trial = true` without a deterministic resolution policy.

UI pattern. Tenant entitlement detail with source, package version, status, allowed intents, effective dates, precedence, and evidence IDs.

Value. Enables reliable SaaS packaging, auditability, migration, renewal, and support investigation.

Risks and tradeoffs. More tables and precedence rules increase operational complexity. Do not store a single mutable "current entitlement" without event evidence. Avoid premature usage-metering or seat rules until product requirements exist.

Implementation path. Add schema without changing access; sync the code manifest into module records with a manifest version; seed package drafts; implement the effective read model; run migration dry-run; review unknown labels and dependency gaps; apply idempotently; keep `requestedModules` as evidence and fallback during observation.

Verification. Prisma validation; migration on a production-like clone; composite-FK and cross-tenant rejection tests; uniqueness and tenant-scope tests; explicit-empty tests; deterministic duplicate-source tests; no-access-loss comparison; precedence and dependency-status tests; time-window tests; idempotent rerun; append-only trigger tests; rollback to legacy read model without schema rollback.

Proposal 3: Product Packages That Start Simple

Business use case. Sell understandable bundles without turning every internal domain into an independently priced product.

Benefiting roles. Customer, sales, customer success, finance, product, and support.

Confirmed current behavior. The repository documents ten candidate packages, but no package is implemented or commercially approved.

Truth owner. Product-owned package policy materialized through versioned database records and validated by a package service.

Boundary. Start with a smaller launch set:

Launch family	Initial content	Position
Platform Base	dashboard, settings, administration	Always present, not sold separately
Operations Core	inventory, sales, reports	Entry operating package
Retail POS add-on	POS and cash drawer, requiring Operations Core	Retail/counter workflow
Finance and Assurance	accounting and finance, with reconciliation, close, and compliance as tiered capabilities	Finance/control family
People and Payroll	presence and payroll, with country/compliance dependencies	Sensitive people workflow
Later add-ons	production, analytics, commercial agents	Launch only after demand and dependency evidence

Reports and analytics must never grant source-module access. `content` remains internal unless a product decision promotes it.

UI pattern. Package comparison and builder for internal product/admin users; tenant owners see only available upgrades, dependencies, impact, and commercial next steps.

Value. Reduces sales complexity, makes support explanations clearer, and preserves room for future enterprise contracts.

Risks and tradeoffs. The documented ten-package matrix is a useful design catalog but too complex to launch without customer evidence. Country-specific pricing, seats, usage limits, and negotiated enterprise terms should not be hard-coded into the first schema.

Implementation path. Validate package families with real customer workflows; define included versus add-on modules; version package composition; add dependency validation; support quote-only prices initially; add pricing and tax behavior only after commercial ownership is clear.

Verification. Package composition tests; dependency closure; no internal module sold alone; source-data filtering tests; package-version immutability; upgrade/downgrade impact preview; contract tests for negotiated overrides.

Proposal 4: Provider-Independent Billing and Provisioning

Business use case. Convert payment/subscription events into reliable internal access without allowing external provider drift or webhook retries to corrupt tenant state.

Benefiting roles. Finance, tenant owner, support, platform operations, security, and product.

Confirmed current behavior. Payment reconciliation has useful provider-event patterns, but no module subscription billing service exists.

Truth owner. Internal subscription state owns commercial status. Internal entitlement state owns runtime access. Providers supply signed evidence only.

Boundary. Add:

- `billing-provider-adapter.service.ts`;
- `billing-provider-event-inbox.service.ts`;
- `tenant-subscription.service.ts`;
- `module-provisioning.service.ts`;
- `billing-entitlement-reconciliation.service.ts`;
- `manual-entitlement-override.service.ts`.

Provider events require signature verification, idempotency, redaction, correlation IDs, payload hashes, normalized event types, processing leases, retries, and a dead-letter path. Provisioning commands require deterministic command IDs and transactional writes of subscription state, grants/constraints, and event evidence.

UI pattern. Billing-entitlement reconciliation workbench showing provider state, internal subscription state, expected entitlements, effective entitlements, drift, age, owner, action, and proof.

Value. Prevents duplicate grants, accidental revocation, webhook-driven outages, and support guesswork.

Risks and tradeoffs. Billing integration is operationally expensive. A manual-invoice workflow may remain necessary for some markets. Build the internal boundary first and add providers through adapters rather than coupling the core model to one vendor.

Implementation path. Implement manual/internal subscriptions first; build idempotent provisioning; add reconciliation; then onboard one provider adapter; run shadow reconciliation before allowing provider-driven transitions; add staged dunning and notification policies.

Verification. Signature-negative tests; duplicate and out-of-order event tests; replay tests; transaction atomicity; reconciliation drift fixtures; provider outage simulation; redaction tests; no direct provider-to-access write test.

Proposal 5: Canonical Access Decision and Surface Registry

Business use case. Ensure direct URLs, actions, APIs, reports, exports, jobs, webhooks, and proofs apply the same tenant module policy before data access or output generation.

Benefiting roles. Every user, plus security, auditors, module owners, and platform engineers.

Confirmed current behavior. Guard seams exist, but mode, order, intent, and unavailable responses vary. The scanner has 55 grandfathered active gaps and incomplete output-surface coverage.

Truth owner. `requireModuleAccess()` plus a versioned surface registry.

Boundary. Each surface declares:

- stable surface key and type;
- owner module and source modules;
- tenant and permission requirements;
- access intent: read, write, export, job, webhook, certify, administer;
- risk level and fresh-auth requirement;
- lifecycle behavior for trial, read-only, suspended, expired, dependency-missing, and rollback;
- denial/partial/redaction behavior;
- audit policy and operational owner.

Recommended decision order is session, tenant scope, module policy, RBAC, fresh auth, maker-checker/consent where required, data access, output redaction, and evidence. Exact order can vary for safe error behavior, but it must be centralized and tested.

UI pattern. Surface-coverage tab with filters by module, surface type, intent, guard, mode, gap, owner, and last verification. Use a detail drawer for evidence and remediation.

Value. Eliminates UI-only security, reduces bespoke guard code, and makes module readiness measurable.

Risks and tradeoffs. A static scanner alone will always have parser limitations. Move toward explicit typed registration for high-risk and output-producing surfaces while retaining scanning as a discovery backstop.

Implementation path. Fix or classify the 55 baseline gaps; add explicit records for reports/exports/jobs/webhooks/proofs/BI; implement the canonical decision service in observe mode; migrate one surface family at a time; add source-module filtering; prohibit new unregistered high-risk surfaces.

Verification. Allow/deny tests before repository access; cross-tenant negative tests; wildcard-RBAC tests; source-module leakage tests; direct URL/action/API parity; read-only intent tests; registry-schema and no-new-gap gates.

Proposal 6: Safe Deactivation, Downgrade, and Retention

Business use case. Stop new work when a module is downgraded or suspended while preserving legally and operationally necessary historical evidence.

Benefiting roles. Tenant owner, finance, accountants, HR/payroll, compliance, auditors, support, and legal stakeholders.

Confirmed current behavior. Lifecycle status types exist, but no durable deactivation state machine or service-owned retention policy exists.

Truth owner. A module lifecycle policy registry plus provisioning/finalizer services. Domain services own the actual retention and write-block rules for their data.

Boundary. Use explicit transitions such as active -> grace/dunning -> read-only -> suspended -> cancelled/archive, with reactivation and rollback. Deactivation requires an impact preview, dependency check, scheduled effective time, job/webhook handling, open-work finalization, retention class, and evidence pack. Never cascade-delete business history because a subscription ends.

UI pattern. Deactivation wizard for owners/support: scope, affected workflows, dependency impact, open work, retained evidence, blocked intents, effective time, approval, and proof. Normal users see calm read-only or unavailable states without billing detail.

Value. Protects data trust and customer continuity while allowing commercial enforcement.

Risks and tradeoffs. Retention rules vary by module and country. A universal read-only promise may be legally wrong or operationally unsafe. Begin with explicit policies for POS/receipts, accounting, payroll, purchasing, and reconciliation.

Implementation path. Define policy per critical module; add preflight read models; implement write blockers and job behavior; add maker-checker for critical modules; create retention grants during downgrade; verify reactivation; add purge only through separate legal-retention workflows.

Verification. State-machine tests; open-transaction fixtures; scheduled job/webhook tests; report/export rules; legal evidence read tests; reactivation tests; maker-checker and fresh-auth tests; no data-deletion assertion.

Proposal 7: Module Workbench and Role-Aware UX

Business use case. Let administrators understand state, risk, action, and proof without making normal users learn billing or entitlement internals.

Benefiting roles. Tenant owner/admin, module owner, finance, support, security, release manager, and normal operators.

Confirmed current behavior. The current page is a permission-protected diagnostic screen with repeated module cards. Shared command-center primitives already include `CommandBriefHeader`, `StatusStrip`, `ActionQueue`, `ProofBadge`, `EvidenceTimeline`, `FilterBar`, and `DetailDrawer`.

Truth owner. `module-workbench-read-model.service.ts`; UI renders its contract and never derives entitlement state.

Boundary. The read model should include package version, subscription status, entitlement source, lifecycle state, allowed/blocked intents, dependency gaps, source modules, surface coverage, billing drift, last decision, evidence IDs, and role-safe next actions.

UI pattern. Replace the card-per-module list with:

1. Command brief: effective control mode, tenant/package, policy version, and last reconciliation.
2. Compact status strip: active, trial, read-only, suspended, would-block, dependency gaps, and registry gaps.
3. Action queue: expiring trials, billing drift, missing dependencies, stale overrides, unregistered enforce call sites, and failed provisioning.
4. Proof strip: entitlement snapshot version, last event, last ratchet, last reconciliation, and rollback readiness.
5. Workbench table: module, state, package/source, allowed intents, dependencies, coverage, last decision, owner, action.

6. Detail drawer: Summary, Access, Lifecycle, Dependencies, Surfaces, Billing, and Evidence tabs.

Separate internal package builder and support override surfaces from the tenant-facing Workbench. Preserve the dark command-center tokens and desktop/mobile parity.

Value. Converts module control from scattered diagnostics into an operational workflow.

Risks and tradeoffs. Building UI before durable services will create polished fiction. Keep unsupported controls read-only and clearly diagnostic until commands are service-backed.

Implementation path. First normalize the current diagnostic page onto shared primitives; then add a service-owned read model; introduce table/drawer navigation; add owner actions only after package/provisioning services exist; add normal-user direct-route states last, after server-side guards are reliable.

Verification. Role-based component tests; safe-copy tests; direct URL parity; loading/error/partial/empty states; mobile and desktop screenshots; accessibility checks; no billing detail for normal users; no UI-derived access decisions.

Proposal 8: Security, Overrides, Audit, and Break-Glass

Business use case. Make high-risk module changes attributable, reviewable, reversible, and resistant to abuse.

Benefiting roles. Security, support, tenant owners, auditors, compliance, release managers, and incident responders.

Confirmed current behavior. Fresh-auth support exists in the shared wrapper. Module would-block events use `AuditLog`. There is no first-class override, maker-checker, break-glass, or immutable lifecycle trail.

Truth owner. Security policy service for authorization controls; append-only entitlement-event service for lifecycle evidence; observability service for access decisions and alerts.

Boundary. Require fresh auth for package changes, suspension/resumption, reconciliation corrections, and overrides. Require maker-checker for critical finance, payroll, accounting, close, and compliance changes. Manual overrides require scope, reason, incident/ticket, start, expiry, approver, review status, and automatic revocation. Break-glass uses a separate permission, short expiry, prominent alerts, session revalidation, and mandatory post-incident review.

Persist every lifecycle change. Persist denials and all overrides. High-volume allowed access decisions can use structured telemetry with bounded retention rather than one database row per request. Use keyed integrity protection or signed evidence bundles only where it adds verifiable value; cryptographic hashes do not replace authorization, encryption, or append-only controls.

UI pattern. Evidence timeline, override queue, expiry countdown, approval drawer, break-glass banner, and post-incident review task.

Value. Improves accountability, incident response, customer trust, and audit readiness.

Risks and tradeoffs. Excessive database logging can become expensive and expose sensitive metadata. Redact payloads, separate operational telemetry from durable lifecycle evidence, and define retention.

Implementation path. Create entitlement event types; correct event names for observe versus deny; add actor/approver/correlation fields; implement fresh-auth override commands; add maker-checker for critical modules; add alerts and review SLA; later add evidence-bundle signing and key rotation if required.

Verification. Unauthorized override tests; self-approval denial; expired override revocation; break-glass alert and review test; append-only mutation tests; redaction tests; audit-write failure alerting; tenant-isolation queries.

Proposal 9: Release Governance and Observability

Business use case. Promote module enforcement gradually with evidence that it can be rolled back without access loss or data leakage.

Benefiting roles. Release manager, security, module owner, support, product, and executive owner.

Confirmed current behavior. Report/warn/fail scanner modes and a saved baseline exist; warn ratchet passes. The gate is not in CI/policy, 55 gaps remain, and no per-surface pilot registry or kill-switch contract is live.

Truth owner. Release governance service/scripts plus a versioned enforcement-policy registry.

Boundary. Use a ladder:

Gate	Promotion condition
0. Inventory	Report current surfaces and classifications.
1. Baseline ratchet	No new ownership, permission, or guard gaps.
2. Registry completeness	High-risk and output surfaces explicitly registered.
3. Durable truth	Package/subscription/entitlement migration proves no access loss.
4. Canonical guard	New/changed surfaces use one access contract.
5. Leakage prevention	Source-module report/export/job/webhook tests pass.
6. Bounded pilot	Named module, surface, cohort, approval, evidence, and kill switch.
7. Module promotion	All module surfaces, UX states, support runbooks, and rollback pass.
8. Broad policy gate	Only after every commercial module has graduated.

Observability should measure allow/would-block/deny by module and surface, policy version, decision latency, cache age, provider drift, provisioning failures, stale overrides, dependency blocks, and rollback state. Never use raw tenant or payment secrets as metric labels.

UI pattern. Module Release Readiness screen with gate status, unresolved gaps, pilot cohort, failure budget, last rollback smoke, owner, and evidence links.

Value. Converts enforcement from a feature flag gamble into controlled operational change.

Risks and tradeoffs. Gates can become ceremonial if evidence is stale or baseline exceptions never expire. Every grandfathered gap needs an owner and review date.

Implementation path. Stabilize the existing baseline; classify false positives; wire warn mode into CI; fail only on new gaps; add explicit output registry; create pilot manifest and kill switch; run shadow decisions; promote one low-risk read surface; expand module by module.

Verification. CI ratchet tests; stale-evidence detection; policy-version assertions; pilot allow/deny/audit tests; rollback smoke; support runbook exercise; synthetic provisioning and provider-drift checks.

Security Control Model

For a protected operation, all applicable controls must pass:

1. Valid, revocable session.
2. Trusted tenant scope.
3. Effective module entitlement for the tenant and intent.
4. User RBAC permission.
5. Fresh authentication for sensitive changes.
6. Maker-checker or consent where required.
7. Domain preconditions and dependency state.
8. Output redaction and source-module policy.
9. Audit/evidence and observability.

Residual risk remains even after these controls. Provider outages, configuration errors, compromised privileged accounts, stale caches, software defects, and incomplete country rules cannot be made impossible. The design should be defense-in-depth, abuse-resistant, auditable, and recoverable, not described as bulletproof.

Practical Roadmap

Phase 0: Truthful control state, 1 to 2 weeks

- Remove ordinary wildcard-role cross-tenant access from POS terminal management.
- Add role-assignment grant ceilings and real server-verifiable step-up evidence.
- Inventory every production enforce/observe call site.
- Centralize mode resolution.
- Make the Module Control Center report effective mixed/pilot state accurately.
- Add per-pilot kill switches and policy versions.
- Keep broad default in observe mode.

Exit: no unregistered enforced surface and support can explain every decision.

Phase 1: Registry and dependency ratchet, 2 to 4 weeks

- Classify the 55 baseline gaps.
- Correct catalog dependency wiring.
- Add first-class reports, exports, jobs, webhooks, proofs, and BI records.
- Wire warn mode into CI; fail only on new high-risk gaps after burn-in.

Exit: every critical surface has owner, module, permission, intent, and unavailable behavior.

Phase 2: Additive entitlement schema and migration, 3 to 5 weeks

- Add versioned module/package, subscription, grant/constraint, and event models.

- Build the effective read model.
- Run dry-run migration from `requestedModules`.
- Preserve legacy full-suite access in observe mode.
- Remove or derive hard-coded session module claims.

Exit: every tenant has explicit effective entitlements with no access-loss delta.

Phase 3: Package and provisioning services, 4 to 6 weeks

- Launch a small package family set.
- Implement manual/internal subscriptions first.
- Add idempotent provisioning and reconciliation.
- Add trial, override, dunning, read-only, suspension, and reactivation commands.

Exit: package changes produce deterministic entitlements and append-only evidence.

Phase 4: Module Workbench and lifecycle UX, 3 to 5 weeks

- Replace card-heavy diagnostics with command brief, status strip, queue, proof strip, table, and drawer.
- Add package/source, dependencies, retention, audit, and reconciliation state.
- Implement normal-user unavailable/read-only experiences.

Exit: every screen answers state, risk, action, and proof from service-owned truth.

Phase 5: Provider adapter and shadow billing, 3 to 6 weeks

- Add one provider inbox/adaptor.
- Reconcile provider, subscription, package, and entitlement states in shadow mode.
- Exercise duplicate, delayed, out-of-order, invalid-signature, and outage paths.

Exit: provider events can be retried safely and never write access directly.

Phase 6: Bounded pilots and module-by-module rollout

- Begin with a low-risk, read-only, explicitly registered surface.
- Require deny, allow, audit, safe UI, support, and rollback evidence.
- Promote one module at a time; enforce writes and outputs last.

Exit: each promoted module has complete surface coverage, operational support, and tested rollback.

Prioritization

Priority	Work	Reason
P0	Close wildcard-role cross-tenant access	Tenant administrators must never become platform operators implicitly.
P0	Add role-assignment grant ceilings and verified step-up	Privileged module commands cannot rely on invitation permission or session age alone.

Priority	Work	Reason
P0	Reconcile mixed enforcement and truthful status	Current UI and runtime behavior disagree.
P0	Protect enforced surfaces with registered policy and rollback	Prevent accidental tenant denial.
P1	Durable entitlement schema and dry-run migration	All commercial behavior depends on trustworthy tenant truth.
P1	Explicit output-surface registry	Prevent report/export/job/webhook leakage.
P1	Correct dependencies and baseline classifications	Current graph cannot support package or deactivation decisions.
P2	Versioned packages and internal subscriptions	Enables commercial operation without provider coupling.
P2	Provisioning, reconciliation, overrides, and lifecycle events	Makes changes reliable and auditable.
P2	Module Workbench read model and table/drawer UI	Gives operators a usable control surface.
P3	First billing provider adapter	Valuable only after the internal boundary is stable.
P3	Advanced pricing, usage metering, and many package variants	Defer until product evidence justifies complexity.

Cases Where Work May Not Be Worth Building Yet

- Do not implement all ten documented package variants before customer and pricing validation.
- Do not add usage metering, seats, proration, coupons, or complex enterprise contracts to the first entitlement schema.
- Do not make every internal domain sellable. Settings, administration, and dashboard are platform foundations.
- Do not hide inactive modules in the sidebar before direct URL and server-side behavior are reliable.
- Do not persist every successful access decision indefinitely. Durable lifecycle evidence and sampled operational telemetry have different purposes.
- Do not enable broad fail-mode enforcement merely because the baseline ratchet passes. It passes with 55 grandfathered gaps.
- Do not connect a billing webhook directly to runtime authorization.
- Do not delete historical business records when a module is deactivated.

Release Verification Matrix

Area	Minimum release evidence
Catalog	Canonical slugs, aliases, dependencies, owner, risk, manifest version

Area	Minimum release evidence
Schema	Prisma validation, migration rehearsal, constraints, idempotency, rollback
Migration	Dry-run/apply parity, unknown-label report, no-access-loss comparison
Package	Version immutability, dependency closure, source-data restrictions
Provisioning	Duplicate/replay/out-of-order tests, transactional event evidence
Access	Tenant, module, RBAC, intent, fresh-auth, maker-checker negative tests
Outputs	Report/export/job/webhook source-module leakage tests
Lifecycle	Trial expiry, dunning, read-only, suspension, cancellation, reactivation
Override	Expiry, approver separation, break-glass alert, post-review
UI	Role-safe states, direct URL parity, mobile/desktop, accessibility
Release	Inventory ratchet, pilot manifest, support runbook, rollback smoke

Final Recommendation

Stoquify should continue investing in the module system, but should treat it as a control-plane program rather than a navigation or pricing feature.

The strongest immediate move is to reconcile actual enforcement behavior with reported control state. After that, build durable entitlement truth and explicit surface ownership before expanding packages, billing, or sidebar behavior. The current catalog, evaluator, Control Center, guard seams, command-center primitives, and inventory ratchet make this achievable without replacing the platform.

Broad rollout should remain blocked until ordinary wildcard roles cannot cross tenant boundaries, invitation role assignment has an enforceable grant ceiling, and sensitive module commands use server-verifiable step-up authentication rather than session age.

The system should be considered ready for broad commercial module enforcement only when:

- every tenant has explicit effective entitlements;
- packages are versioned and subscription state is internal;
- provider events cannot directly grant or revoke access;
- every high-risk and output surface is registered;
- one canonical access decision service is used before data access;
- read-only, suspended, expired, and unavailable behavior is service-owned;
- lifecycle changes are append-only and auditable;
- normal and admin UI states are role-safe;
- every promoted module has deny, leakage, support, and rollback evidence.

Until then, Stoquify has a strong module-governance foundation and several real enforcement pilots, but not yet a complete enterprise module control plane.

Evidence References

- `services/modules/module-control-contracts.ts`
- `services/modules/module-catalog.service.ts`
- `services/modules/module-entitlement.service.ts`
- `services/_shared/protect.ts`
- `lib/security/server-authz.ts`
- `lib/security/auth-session.ts`
- `actions/modules/module-control.actions.ts`
- `app/[locale]/(dashboard)/dashboard/settings/modules/page.tsx`
- `config/sidebar.ts`
- `components/dashboard/Sidebar.tsx`
- `components/dashboard/primitives/command-center-primitives.tsx`
- `prisma/schema.prisma`
- `scripts/module-surface-inventory.js`
- `scripts/__tests__/module-surface-inventory.test.js`
- `services/modules/__tests__/module-entitlement.service.test.ts`
- `actions/modules/__tests__/module-control.actions.test.ts`
- `docs/modules/AQSTOQFLOW_COMPLETE_MODULE_SYSTEM_ENTERPRISE_ROADMAP_2026-07-12.md`
- `docs/modules/AQSTOQFLOW_MODULE_ENTITLEMENT_SCHEMA_AND_MIGRATION_PLAN_2026-07-12.md`
- `docs/modules/AQSTOQFLOW_MODULE_PACKAGE_STRATEGY_2026-07-12.md`
- `docs/modules/AQSTOQFLOW_MODULE_BILLING_PROVISIONING_BOUNDARY_2026-07-12.md`
- `docs/modules/AQSTOQFLOW_MODULE_WORKBENCH_UX_STATES_2026-07-12.md`
- `docs/modules/AQSTOQFLOW_MODULE_RELEASE_GATES_AND_ROLLBACK_2026-07-12.md`
- `what-next/module-surface-registry-baseline-2026-07-12.json`
- `what-next/module-surface-inventory.json`
- `graphify-out/GRAPH_REPORT.md` and `graphify-out/manifest.json` as stale secondary context only

Scope and Non-Claims

- No application code, schema, runtime access behavior, package state, or entitlement state was changed.
- No database migration or provider integration was executed.
- The proposed package families and timelines are planning recommendations, not approved commercial commitments.
- The report does not claim the system is bulletproof or broadly production-ready for module enforcement.
- Existing worktree changes were preserved and not cleaned, staged, committed, or pushed.